

Student Storefront

Turning campus WhatsApp sales into a verified marketplace

Pre-defense goal: show product value, technical uncertainty solved, and a demo path that works in 10 minutes.

Problem: WhatsApp sells, but does not organize

Current behavior

- Items posted in scattered groups
- Sold items stay visible
- Search depends on chat history
- Buyer trust depends on guessing

Cost to students

- Repeated "still available?" messages
- Lost posts after a few hours
- No structured categories or status
- No campus-only seller verification

Product thesis: keep WhatsApp reach, add marketplace structure

Buyers get

- Public PWA storefront
- Search, category filter, sorting
- Multi-photo listings
- One-click WhatsApp contact

Sellers get

- Verified `@constructor.university` identity
- Listing creation and management
- Mark sold / delete from profile
- WhatsApp bridge for lower-friction posting

What works now

Area	Current implementation
Storefront	Alpine.js PWA served by Spring Boot
Backend	Kotlin Spring Boot, JPA, PostgreSQL
Auth	JWT login/register with campus email validation
Listings	Create, upload images, browse, filter, sort, mark sold
Trust	Seller profiles, phone/email contact, verified campus domain
Integration	Node WhatsApp bot, Gemini parsing, Cloudinary uploads

Demo checkpoint 1: buyer experience

Show in the app

1. Open **Campus Store** listings
2. Search for an item
3. Filter by category and sort by price/newest
4. Open seller/contact affordances
5. Save a favourite if logged in

Claim: a buyer can find active campus inventory faster than scrolling WhatsApp history.

Demo checkpoint 2: seller workflow

Show in the app

1. Log in or register with `@constructor.university`
2. Create listing with title, category, price, description
3. Upload multiple images and preview them
4. Visit profile dashboard
5. Mark item sold or delete listing

Claim: listing state becomes explicit; buyers stop seeing stale posts.

Technical challenge 1: two entry points, one source of truth

Uncertainty

How do we keep WhatsApp convenience without making WhatsApp the database?

Decision

Spring Boot owns listings and auth. The bot only translates chat interactions into backend API calls.

Result: web actions and bot actions converge on the same `Post` , `Seller` , and `PostMedia` model.

Technical challenge 2: WhatsApp QR login safely bridges devices

```
Browser -> POST /api/auth/whatsapp/session  
Backend -> sessionId + wa.me login token  
Phone    -> sends login:<token> to bot  
Bot      -> POST /api/auth/whatsapp/confirm with X-Bot-API-Key  
Browser  -> polls session, claims one-time token  
Backend  -> returns JWT
```

Risk handled: tokens expire, claim token is single-use, bot endpoint uses a shared secret.

Demo checkpoint 3: WhatsApp integration

Preferred live demo

1. Click WhatsApp login
2. Show QR / `wa.me` deep link
3. Send prefilled token to bot
4. Watch browser poll and receive JWT

Fallback demo

Show the sequence diagram and API responses from `docs/REPOMAP.md`.

Rule: if the bot or WhatsApp session fails, spend one sentence and move on.

Technical challenge 3: images and structured listings

Product need

Listings without images are low-trust and low-conversion.

Engineering path

- Multipart upload endpoint
- Up to 10 images
- Cover image ordering
- Cloudinary-hosted media URLs
- Gemini-assisted bot parsing

Learning: media upload turns a simple CRUD app into a workflow with validation, storage, ordering, and failure handling.

Evidence: implemented and tested vertical slices

Working slices

- Public browse and search
- Authenticated create listing
- Profile listing management
- Favourites sync
- WhatsApp QR login endpoints

Test coverage focus

- `WhatsAppQrLoginServiceTest` : service branches
- `WhatsAppQrLoginControllerTest` : HTTP flow with Testcontainers
- Validation in frontend and backend DTO paths

Known gaps before final defense

Gap	Why it matters	Next action
Public <code>POST /api/sellers</code>	Admin bypass risk	Restrict to admin role
Public phone lookup	PII exposure risk	Require <code>X-Bot-Api-Key</code>
Live bot dependency	Demo fragility	Keep recording/API fallback
User validation	Product value evidence	Run small campus test with real listings

Final defense focus

What we want the jury to remember

Student Storefront is a campus marketplace where WhatsApp remains the social channel, while the backend becomes the trusted inventory system.

Next sprint priorities

- Lock down bot-only/admin endpoints
- Stabilize demo data and fallback recording
- Validate with real student listings
- Polish mobile-first PWA flow